40

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

Routes vs Ingress: OpenShift Trafiği Nasıl Dışa Açar?

OpenShift Routes vs K8s Ingress: Edge TLS Demystified

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes dünyası henüz Ingress'i icat etmemişken, Red Hat mühendisleri 'Route' mekanizmasını geliştirmişti! K8s'te Ingress yazmak için önce gidip NGINX controller kurmanız, DNS kayıtlarıyla boğuşmanız gerekir. OpenShift'te ise tek bir komut verirsiniz (`oc expose svc coffee-service`), sistem size saniyesinde geçerli bir HTTPS adresi (`coffee-service-coffee-shop.apps.mycluster.com`) hediye eder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Years before Kubernetes standardized Ingress, Red Hat engineered OpenShift Routes! In plain K8s, exposing an app requires installing an ingress controller, procuring domain names, and mapping DNS records. In OpenShift, you run ONE command (`oc expose svc coffee-service`), and OpenShift instantly creates a public, TLS-secured wildcard domain!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

OpenShift Route, küme girişindeki HAProxy tabanlı Router pod'ları tarafından yönetilir. Üç farklı TLS sonlandırma stratejisi sunar: 1) `Edge`: SSL sertifikası Route (HAProxy) seviyesinde sonlandırılır, arkadaki Spring Boot pod'una düz HTTP olarak iletilir (en yaygın ve hızlı model). 2) `Passthrough`: Şifrelenmiş SSL trafiği hiç açılmadan doğrudan pod'un içine kadar iletilir (uçtan uca şifreleme). 3) `Re-encrypt`: Route sertifikayı açar, inceler ve pod'a giderken farklı bir iç sertifikayla yeniden şifreler.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OpenShift Routes are fulfilled by HAProxy routers. Three TLS termination strategies are supported: 1) `Edge`: TLS terminates at the HAProxy router, proxying plain HTTP to backend pods (highest performance), 2) `Passthrough`: Encrypted TLS packets pass through untouched directly to the pod's embedded SSL engine, 3) `Re-encrypt`: The router decrypts external traffic, inspects it, and re-encrypts using an internal certificate before reaching the pod.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# coffee-route.yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: coffee-route
  namespace: coffee-shop
spec:
  to:
    kind: Service
    name: coffee-service
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect # HTTP isteklerini otomatik HTTPS'e yönlendir!

# CLI ile tek komutta Route üretmek:
# oc expose service coffee-service --port=http
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

OpenShift aynı zamanda standart Kubernetes `Ingress` kaynaklarını da destekler (arka planda otomatik Route'a dönüştürür). Ancak kurumsal sertifika yönetimi için Route tanımlamak çok daha güçlü ve pratiktir.

🇬🇧 WATCH OUT

OpenShift automatically translates K8s Ingress objects into Routes under the hood, but native Routes provide finer-grained TLS and timeout controls.

🎯 Özet Çıkarımlar

- Route, OpenShift'in yerleşik ve otomatik DNS destekli dışa açma mekanizmasıdır.
- `Edge`, `Passthrough` ve `Re-encrypt` ile esnek TLS mimarileri kurulur.
- HTTP trafiği `Redirect` politikasıyla otomatik olarak güvenli HTTPS'e yükseltilir.

🎯 Key Takeaways

- Routes provide out-of-the-box wildcard DNS and Layer 7 external routing.
- Flexible TLS termination models satisfy rigorous enterprise security compliance.
- Enforces automated HTTP-to-HTTPS redirects with a single line of YAML.